

Experiment 10
Print messages in 16*2 LCD display

Name: Ashutosh Bajpayee
CDSID: abajpaye



Table of Contents

Sl. No.	Topic	Page No.
1	Introduction	3
2	Scope	3
3	Prerequisite	3
4	Connection Diagram	4
5	Working Principle	4
6	Application Code	5
7	Hardware Image	5
8	Appendix	6

Introduction

A 16×2 LCD display is a widely used output device in embedded systems for showing alphanumeric characters. It is based on the HD44780 controller, which supports communication in both 8-bit and 4-bit modes. In this experiment, the LCD is interfaced with the Raspberry Pi Pico using 4-bit mode to reduce the number of GPIO pins required, and a predefined message (name) is displayed on the screen.

Scope

The scope of this experiment is to understand how an LCD is interfaced with a microcontroller, to learn GPIO control along with the required timing constraints, to implement the 4-bit communication protocol, and to display user-defined messages on the LCD.

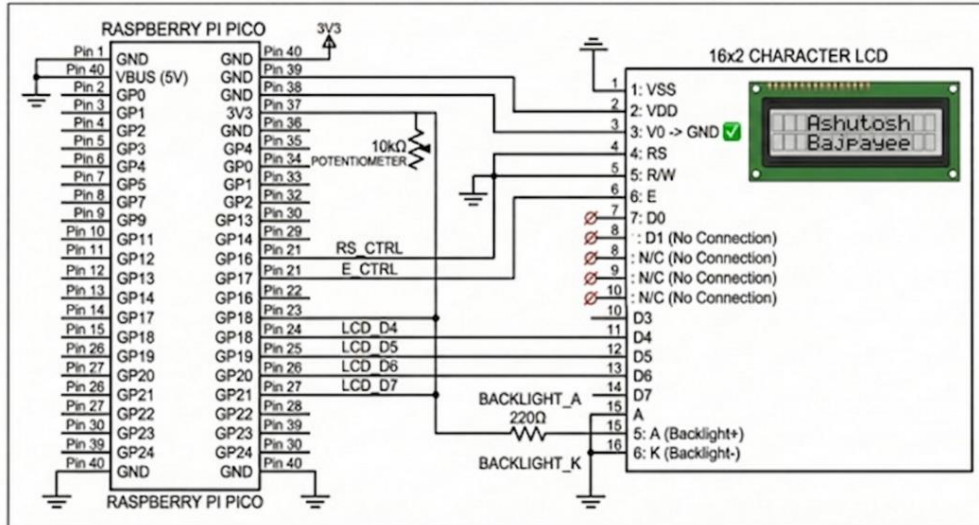
Prerequisite

Hardware: Raspberry Pi Pico microcontroller, USB connector, breadboard, and a 16×2 LCD display.

Software: Visual Studio Code, Pico SDK, CMake, and GCC toolchain.

Connection Diagram

CONNECTION DIAGRAM: RASPBERRY PI PICO TO 16x2 LCD (4-BIT MODE, FIXED CONTRAST, NO POTENTIOMETER).



Working Principle

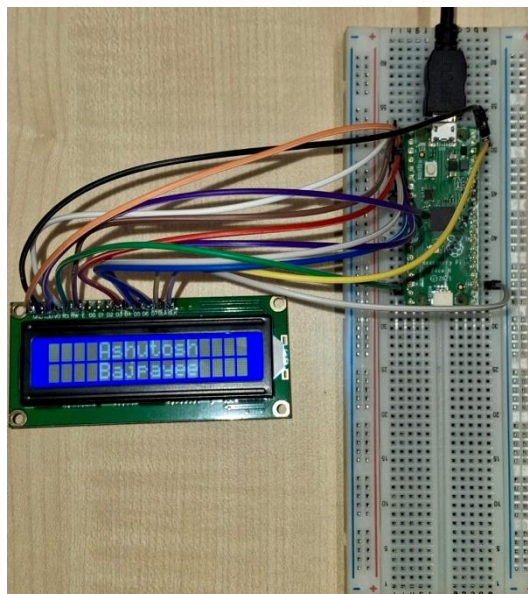
- The LCD operates using the HD44780 controller, which receives commands and data through GPIO pins.
- The RS (Register Select) pin determines the mode of operation: 0 for command mode and 1 for data mode.
- The Enable (E) pin uses a HIGH-to-LOW pulse to trigger data transfer to the LCD.
- In 4-bit mode, data is transmitted in two 4-bit parts (nibbles), reducing the number of required GPIO pins.
- During initialization, the LCD is configured for 4-bit mode, 2-line display, and appropriate cursor settings.
- Contrast control is achieved by connecting the V0 pin to GND, providing a fixed contrast level.

Application code

```
C Assignment_10.c > main()
1  #include "pico/stdlib.h"
2
3  #define RS 0
4  #define EN 1
5  #define D4 2
6  #define D5 3
7  #define D6 4
8  #define D7 5
9
10 void lcd_send_nibble(uint8_t nibble) {
11     gpio_put(D4, (nibble >> 0) & 1);
12     gpio_put(D5, (nibble >> 1) & 1);
13     gpio_put(D6, (nibble >> 2) & 1);
14     gpio_put(D7, (nibble >> 3) & 1);
15     gpio_put(EN, 1);
16     sleep_us(1);
17     gpio_put(EN, 0);
18     sleep_us(100);
19 }
20
21 void lcd_send_byte(uint8_t data, int rs) {
22     gpio_put(RS, rs);
23     lcd_send_nibble(data >> 4);
24     lcd_send_nibble(data & 0x0F);
25 }
26
27 void lcd_cmd(uint8_t cmd) {
28     lcd_send_byte(cmd, 0);
29     sleep_ms(2);
30 }
31
32 void lcd_char(char c) {
33     lcd_send_byte(c, 1);
34 }
35
36 void lcd_string(const char *str) {
37     while (*str) {
38         lcd_char(*str++);
39     }
40 }
```

```
C Assignment_10.c > ...
41
42 void lcd_init() {
43     gpio_init(RS); gpio_set_dir(RS, GPIO_OUT);
44     gpio_init(EN); gpio_set_dir(EN, GPIO_OUT);
45     gpio_init(D4); gpio_set_dir(D4, GPIO_OUT);
46     gpio_init(D5); gpio_set_dir(D5, GPIO_OUT);
47     gpio_init(D6); gpio_set_dir(D6, GPIO_OUT);
48     gpio_init(D7); gpio_set_dir(D7, GPIO_OUT);
49
50     sleep_ms(50);
51
52     lcd_send_nibble(0x03);
53     sleep_ms(5);
54     lcd_send_nibble(0x03);
55     sleep_ms(5);
56     lcd_send_nibble(0x03);
57     sleep_ms(5);
58     lcd_send_nibble(0x02);
59
60     lcd_cmd(0x28);
61     lcd_cmd(0x0C);
62     lcd_cmd(0x06);
63     lcd_cmd(0x01);
64 }
65
66 int main() {
67     stdio_init_all();
68     lcd_init();
69     lcd_cmd(0x80 + 4);
70     lcd_string("Ashutosh");
71     lcd_cmd(0xC0 + 4);
72     lcd_string("Bajpayee");
73     while (1);
74 }
```

Hardware Image



Appendix

- The experiment uses a Raspberry Pi Pico, 16×2 LCD display, breadboard, and connecting wires.
- GPIO pins are used to send commands and data to the LCD through the HD44780 controller.
- The LCD operates in 4-bit mode, reducing the number of required connections.
- Proper initialization ensures correct display configuration and cursor settings.
- A predefined message is displayed successfully, confirming correct interfacing and operation.